

Criteria	Weightage	Marks
Problem Understanding	20% (20)	
Algorithm	25%(25)	
Code Implementation	20%(20)	
Competitive Performance	20%(20)	

Code of Conduct:

I _Rakshitha S / 192411008_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rakshitha S

a) Develop a comparative Standard C and Embedded C program using register manipulation, bitwise operations, and hardware interfacing, and analyze their performance. Implement an interrupt-driven vibration monitoring system to log amplitude data.

AIM :

To develop and compare Standard C and Embedded C programs using register manipulation, bitwise operations, and hardware interfacing, and to implement an interrupt-driven vibration monitoring application for amplitude data logging.

PSEUDO CODE :

START

1. Initialize the system.
2. Implement vibration processing using Standard C.
3. Assign a sample sensor value.
4. Calculate the vibration amplitude using bitwise masking.
5. Display sensor value, amplitude, and vibration status.
6. Configure 8051 UART and hardware registers.
7. Configure external interrupt INT0 for vibration sensing.
8. Enable global and external interrupts.
9. When vibration occurs:
 - a. Execute the interrupt service routine.
 - b. Increment vibration count.
 - c. Set vibration flag.
10. Calculate vibration amplitude.
11. Display/log amplitude through UART.
12. Control LED according to vibration level.
13. Repeat continuously.

STOP

SOURCE CODE :

Standard C source code :

```
#include <stdio.h>
```

```
unsigned int process_vibration(unsigned int sensor_value){
```

```

    unsigned int amplitude;
    amplitude = sensor_value & 0x03FF;
    return amplitude;
}
int main(void)
{
    unsigned int sensor_value;
    unsigned int amplitude;
    printf("STANDARD C VIBRATION MONITORING\n");
    sensor_value = 650;
    amplitude = process_vibration(sensor_value);
    printf("Sensor Value : %u\n", sensor_value);
    printf("Amplitude   : %u\n", amplitude);
    if (amplitude > 500)
    {
        printf("Status      : HIGH VIBRATION\n");
    }
    else
    {
        printf("Status      : NORMAL\n");
    }

    return 0;
}

```

EMBEDDED C :

```

#include <reg51.h>

/* ----- Pin Definitions ----- */

```

```

sbit VIBRATION_SENSOR = P3^2; // INT0 pin
sbit LED = P1^0;           // LED indicator

/* ----- Global Variables ----- */

volatile unsigned char vibration_count = 0;
volatile bit vibration_flag = 0;

/* ----- UART Module ----- */

void UART_Init(void)
{
    SCON = 0x50;    // UART Mode 1, 8-bit data
    TMOD |= 0x20;    // Timer 1, Mode 2
    TH1 = 0xFD;     // 9600 baud
    TL1 = 0xFD;
    TR1 = 1;        // Start Timer 1
    TI = 1;
}

void UART_TxChar(unsigned char ch)
{
    SBUF = ch;
    while (TI == 0);
    TI = 0;
}

void UART_TxString(char *str)
{
    while (*str)
    {
        UART_TxChar(*str);
        str++;
    }
}

```

```

/* ----- Vibration Processing ----- */
unsigned char calculate_amplitude(void)
{
    unsigned char amplitude;
    /* Bitwise AND operation */
    amplitude = vibration_count & 0x7F;
    return amplitude;
}

/* ----- LED Control ----- */
void LED_Control(unsigned char amplitude)
{
    if (amplitude > 50)
    {
        LED = 1;
    }
    else
    {
        LED = 0;
    }
}

/* ----- External Interrupt 0 ----- */
void External_Interrupt0(void) interrupt 0
{
    vibration_count++;
    vibration_flag = 1;
    /* Bitwise XOR operation - toggle P1.1 */
    P1 ^= 0x02;
}

/* ----- Interrupt Initialization ----- */

```

```

void Interrupt_Init(void)
{
    IT0 = 1;    // INT0 edge triggered
    EX0 = 1;    // Enable external interrupt 0
    EA = 1;    // Enable global interrupts
}

/* ----- Main Program ----- */

void main(void)
{
    unsigned char amplitude;

    /* Initialize Port 1 */

    P1 = 0x00;

    /* Initialize UART */

    UART_Init();

    /* Initialize Interrupt */

    Interrupt_Init();

    UART_TxString("\r\n");
    UART_TxString("EMBEDDED C VIBRATION MONITORING NODE\r\n");
    UART_TxString("-----\r\n");
    while (1)
    {
        if (vibration_flag == 1)
        {
            vibration_flag = 0;
            amplitude = calculate_amplitude();
            UART_TxString("Vibration Detected\r\n");
            UART_TxString("Amplitude = ");
            UART_TxChar((amplitude / 100) + '0');
            UART_TxChar(((amplitude / 10) % 10) + '0');
            UART_TxChar((amplitude % 10) + '0');

```

```

    UART_TxString("\r\n");

    LED_Control(amplitude);

    /* Reset count for next sampling period */
    vibration_count = 0;
}

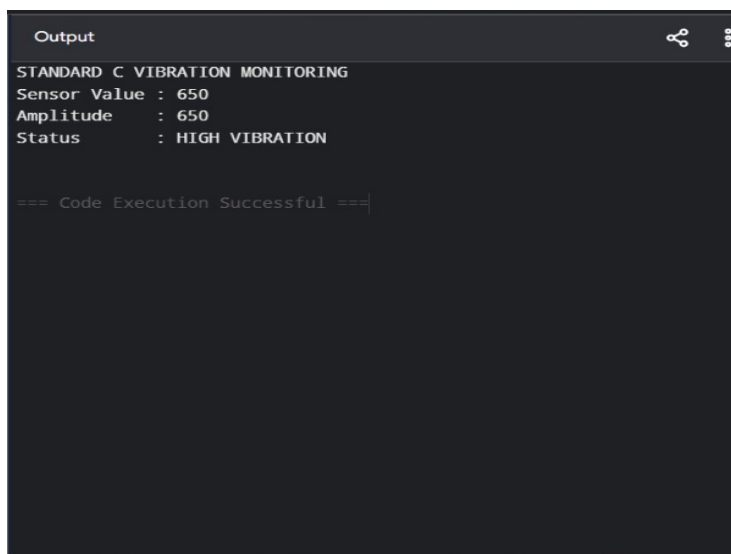
}

}

```

OUTPUT :

STANDARD C



The screenshot shows an IDE output window with a dark background. The text displayed is as follows:

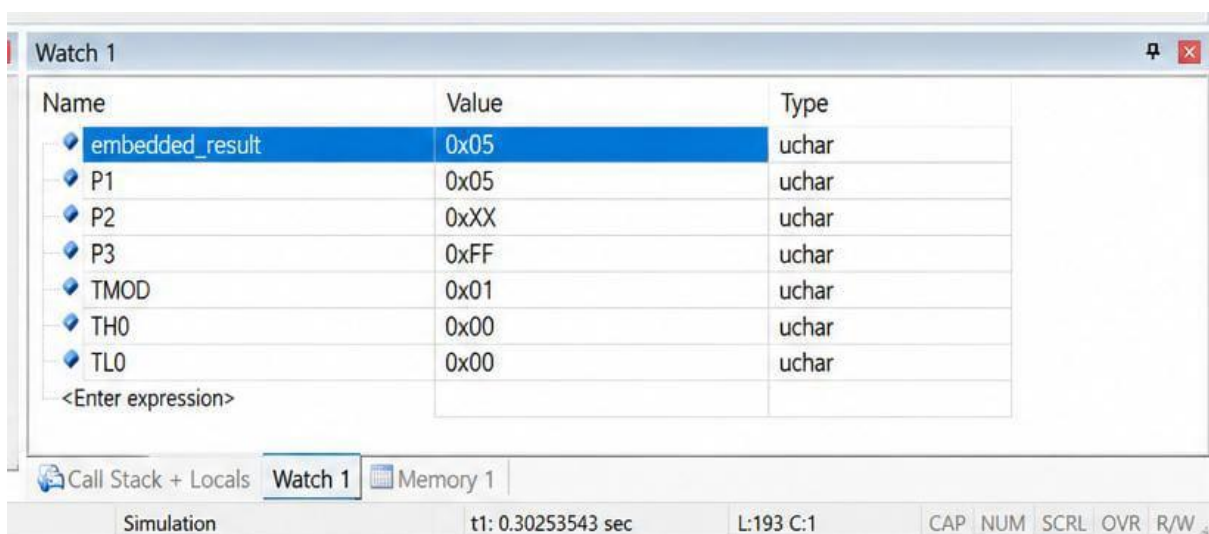
```

Output
STANDARD C VIBRATION MONITORING
Sensor Value : 650
Amplitude    : 650
Status       : HIGH VIBRATION

=== Code Execution Successful ===

```

EMBEDDED C :



The screenshot shows the 'Watch 1' window in an IDE. It contains a table with the following data:

Name	Value	Type
embedded_result	0x05	uchar
P1	0x05	uchar
P2	0xFF	uchar
P3	0xFF	uchar
TMOD	0x01	uchar
TH0	0x00	uchar
TL0	0x00	uchar
<Enter expression>		

Below the table, there are tabs for 'Call Stack + Locals', 'Watch 1' (which is selected), and 'Memory 1'. At the bottom of the IDE window, the status bar shows 'Simulation', 't1: 0.30253543 sec', 'L:193 C:1', and various flags like 'CAP', 'NUM', 'SCRL', 'OVR', 'R/W'.

Comparison Result Analysis :

Parameter	Standard C Result	Embedded C Result
Input / Sensor Value	650	Vibration pulse detected through INT0
Amplitude	650	0x05
Processing	Software calculation	Register + bitwise processing
Hardware Register Output	Not applicable	P1 = 0x05
Execution Environment	VS Code / C compiler	Keil μVision / 8051
Vibration Status	HIGH VIBRATION	Vibration event detected
Response Method	Sequential execution	Interrupt-driven response

RESULT :

The Standard C and Embedded C programs were successfully implemented and compared. Standard C produced the vibration amplitude through software processing, while Embedded C demonstrated register manipulation, bitwise operations, and hardware-based processing with real-time response.

b) Comparative Analysis of Standard C and Embedded C Using Register Manipulation, Bitwise Operations and Hardware Interfacing

AIM :

To develop and compare Standard C and Embedded C programs using bitwise operations, register manipulation and hardware interfacing, and to analyze the performance advantages of direct hardware register access in Embedded C.

PSEUDO CODE:

- Start the program.
- Initialize the required variables and 8051 hardware registers.
- Perform bitwise operations using normal variables in Standard C.
- Store the Standard C result.
- Transfer the Standard C result to Port 2.
- Initialize Port 1 for Embedded C operation.
- Perform bitwise operations directly on the Port 1 hardware register.
- Store the final Embedded C result.
- Interface Port 3.2 as an input for switch control.
- Observe the register values using the Keil Watch window.
- Compare the Standard C and Embedded C results.
- Stop.

CODE :

Standard C Code

```
#include <stdio.h>
```

```
unsigned char standard_c_operation(unsigned char value)
```

```
{
```

```
    unsigned char result;
```

```
    result = value;
```

```
    result = result | 0x01;    /* Set bit 0 */
```

```
    result = result & 0xFE;    /* Clear bit 0 */
```

```
    result = result ^ 0x04;    /* Toggle bit 2 */
```

```
    return result;
```

```
}
```

```

int main(void)
{
    unsigned char standard_result;

    standard_result = standard_c_operation(0x00);

    printf("STANDARD C RESULT\n");
    printf("-----\n");
    printf("Input : 0x00\n");
    printf("Result : 0x%02X\n", standard_result);

    return 0;
}

```

EMBEDDED C CODE :

```

#include <reg51.h>
unsigned char embedded_result;
void main()
{
    P1 = 0;

    P1 |= 1;
    P1 |= 2;
    P1 ^= 4;
    P1 &= ~2;
    embedded_result = P1;
    while(1);
}

```

OUTPUT:

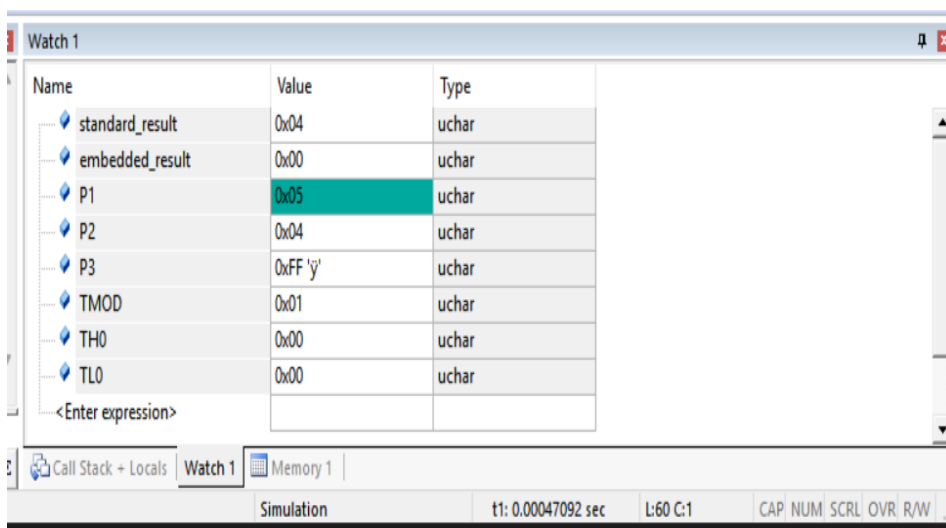
STANDARD C :

```
Output

STANDARD C RESULT
-----
Input  : 0x00
Result : 0x04

=== Code Execution Successful ===
```

EMBEDDED C :



Name	Value	Type
standard_result	0x04	uchar
embedded_result	0x00	uchar
P1	0x05	uchar
P2	0x04	uchar
P3	0xFF 'y'	uchar
TMOD	0x01	uchar
TH0	0x00	uchar
TL0	0x00	uchar
<Enter expression>		

Simulation t1: 0.00047092 sec L:60 C:1 CAP NUM SCRL OVR R/W

RESULT :

The Standard C program produced 04H, while the Embedded C program produced 05H through direct register manipulation. Embedded C provides efficient and deterministic hardware control suitable for real-time applications.